

Lezione 12

12.1 L'ALGORITMO TOPOLOGICAL SORT

Sia $G = (V, E)$ un grafo.

DEF.: Un **ordinamento topologico** di G è un ordinamento lineare $<$ dei suoi vertici tale che

$$(u, v) \in E \Rightarrow u < v.$$

L'**algoritmo TOPOLOGICAL SORT** prende in input un grafo orientato aciclico (DAG, direct acyclic graph) $G = (V, E)$ e restituisce in output un ordinamento topologico di G .

OSS.: È definito solo su DAG.

però temporale
a cui termina
l'esplorazione di v

TOPOLOGICAL SORT

1. invoca DFS su G per calcolare il finish-time v.f., $\forall v \in V$;
2. al termine dell'esplorazione di ogni vertice, inseriscilo in una lista collegata $\mathcal{L}$;
3. return la lista $\mathcal{L}$.

Il tempo computazionale di TOPOLOGICAL SORT è $\Theta(|V| + |E|)$ dato che DFS impiega $\Theta(|V| + |E|)$ e l'inserimento di un vertice nella lista impiega $O(1)$.

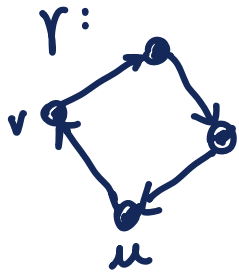
Abbiamo a essere sicuri che TOPOLOGICAL SORT restituisce un ordinamento topologico?

Proviamo dal seguente

Lemma 1: $G=(V,E)$ è un DAG $\iff$ DFS(G) non genera archi all'indietro.

PROOF: ($\implies$): Supponiamo che una DFS su G produce un arco all'indietro (u,v) . Allora, v è un predecessore di u nella foresta DF G_π . Quindi, G contiene un ciclo. * Assurdo perché G è un DAG.

($\impliedby$): Supponiamo che G contenga un ciclo γ e facciamo vedere che in tal caso una DFS genererebbe un arco all'indietro. Sia v il primo vertice in γ ad essere scoperto dalla DFS. Al momento v.d (scoperta di v) i vertici di γ formano un cammino bianco da v al predecessore di v in γ che chiameremo u .

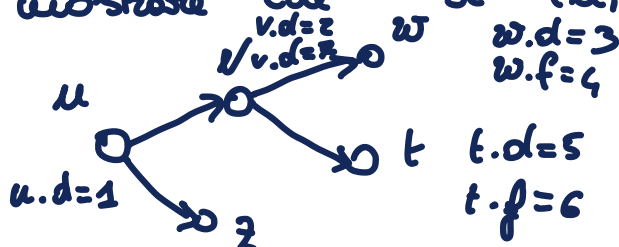


Allora per il WHITE PATH THEOREM, u è discendente di v nella foresta $G_\pi \implies (u,v)$ è un arco all'indietro. *

THM (CORRETTEZZA DI TOPOLOGICAL SORT): $G=(V,E)$ un DAG.

TOPOLOGICAL SORT (G) produce un ordinamento topologico di G .

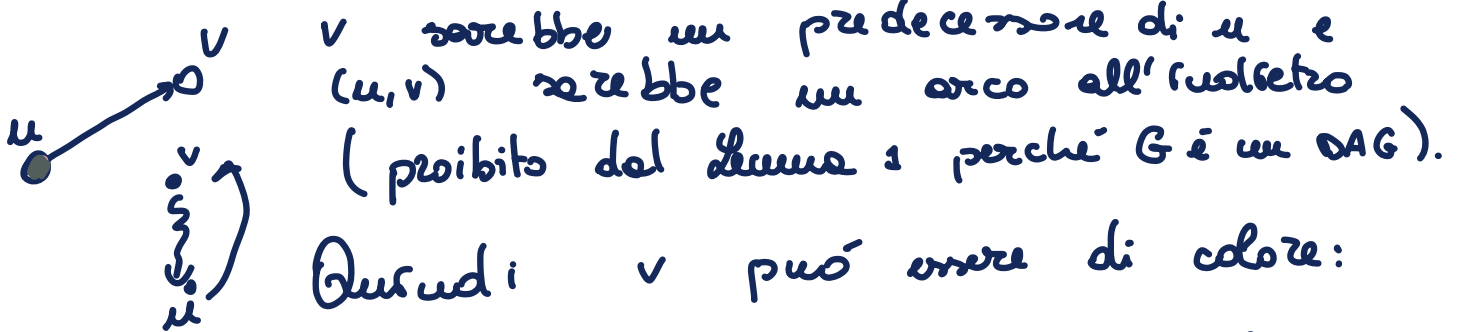
Proof: Per mostrare la correttezza di TOPOLOGICAL SORT è sufficiente mostrare che se $(u,v) \in E$ allora $u < v \iff$



$$\boxed{v.f < u.f}$$

dobbiemo provare questo.

Per mostrarlo osserviamo che quando DFS esplora u , il vertice v non può essere già finito, altrimenti



- Quindi v può essere di colore:
- bianco $\Rightarrow v$ diventa un discendente di u
 $\Rightarrow v.f < u.f.$
 - nero $\Rightarrow v$ è già stato scoperto ed esplorato
 $\Rightarrow v.f$ è già stato determinato
 quindi $v.f < u.f.$ $\square$

12.2 IL PROBLEMA SINGLE-SOURCE SHORTEST PATH

Consideriamo il seguente problema:

SINGLE-SOURCE SHORTEST PATH

- input:
- un grafo $G=(V,E)$;
 - un vertice $s \in V$, chiamato origine;
 - una funzione di peso sugli archi $w: E \rightarrow \mathbb{R}$.

goal: Trovare un cammino minimo da s a v , $\forall v \in V$,
 ovvero $p := \langle \underset{s}{u_0}, u_1, \dots, \underset{v}{u_n} \rangle$ tale che $(u_{i-1}, u_i) \in E$
 e inoltre vogliamo che $w(p) := \sum_{i=1}^n w(u_{i-1}, u_i)$
 sia minimo.

NOTAZIONE:

$$\delta(s, v) := \begin{cases} \min_{s \xrightarrow{E} v} w(p) = \min_{\substack{u_0 = s \\ u_n = v}} \sum_{i=1}^n w(u_{i-1}, u_i) \\ \infty \end{cases}$$

se non esiste alcun cammino da s a v .

Oss: Ci potrebbero essere dei pesi negativi.

- Se G non contiene cicli di peso negativo raggiungibili da s , allora $\delta(s, v)$ è ben definito per ogni v .
- Se G contiene un ciclo di peso negativo raggiungibile da s , allora $\delta(s, v)$ non è ben definito, infatti posso sempre trovare un cammino di peso inferiore ripetendo il ciclo. Pertanto, in tal caso definiremo $\delta(s, v) = -\infty$.

12.3 L'ALGORITMO DI BELLMAN-FORD

Risolve SINGLE-SOURCE SHORTEST PATH nel caso generale su cui i pesi degli archi possono essere negativi ($w: E \rightarrow \mathbb{R}$).

Dato un grafo $G=(V,E)$ orientato, $s \in V$ vertice sorgente e $w: E \rightarrow \mathbb{R}$, l'algoritmo Bellman-Ford restituisce un valore booleano: TRUE o FALSE. FALSE significa che esiste un ciclo di peso negativo raggiungibile dalla sorgente. TRUE significa che $\delta(s,v)$ è ben definito $\forall v \in V$ e in tal caso è possibile ricostruire un cammino minimo da s a v , $\forall v \in V$.

BELLMAN-FORD (G, s, w)

- ```

1. INITIALIZE-SINGLE-SOURCE (G, s)
2. for i = 1 to |V| - 1
3. for (u, v) ∈ E
4. RELAX (u, v, w)
5. for each (u, v) ∈ E
6. if v.d > u.d + w(u, v)
7. return FALSE
8. return TRUE

```

Θ(v)K  
αλφ  
αδγ

## INITIALIZE-SINGLE-SOURCE( $G, s$ )

1. for each  $v \in V$
2.  $v.d = \infty$
3.  $v.\pi = NIL$
4.  $s.d = 0$

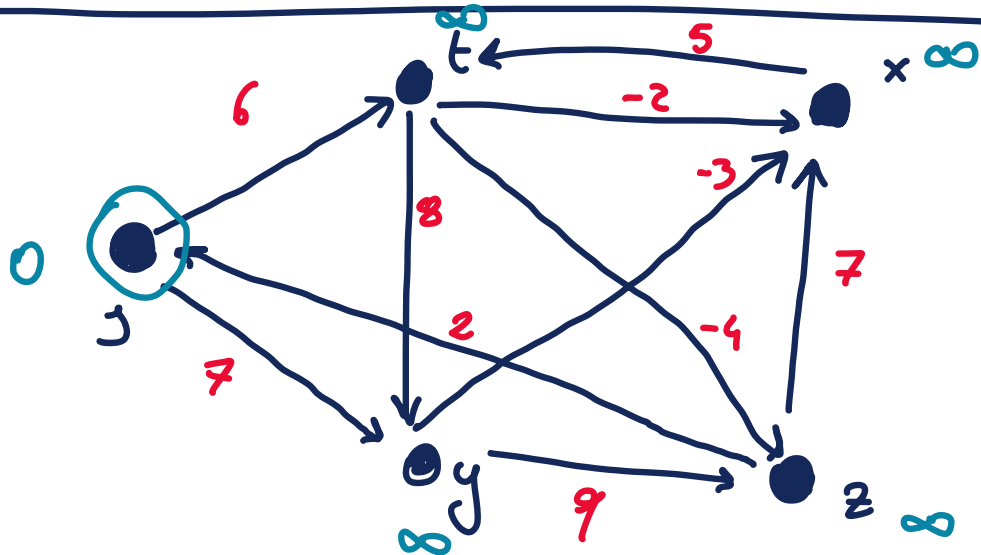
RELAX(u, v, w)

1.  $\frac{d}{dt} v.d > u.d + \omega(u, v)$
2.  $v.d = u.d + \omega(u, v)$
3.  $v.\pi = u$

L'algoritmo Bellman-Ford richiede  $O(|V|^2 + |V||E|)$  - Time,  
quando il grafo è rappresentato dalle sue liste di adiacenze.  
Se  $|E| = \Omega(|V|) \leadsto O(|V||E|)$ .

In molti, spesso sono sufficienti meno di  $|V|-1$  ripetizioni del ciclo alle regole 3-5.

ESEMPIO:



Inizializzazione

$$t.d = \infty$$

$$t.\pi = \text{NIL}$$

$$x.d = \infty$$

$$x.\pi = \text{NIL}$$

$$y.d = \infty$$

$$y.\pi = \text{NIL}$$

$$z.d = \infty$$

$$z.\pi = \text{NIL}$$

$$s.d = 0$$

$$s.\pi = \text{NIL}$$

$$|V|-1 = 5-1 = 4$$

①  $(u,v) : v.d > u.d + w(u,v)$

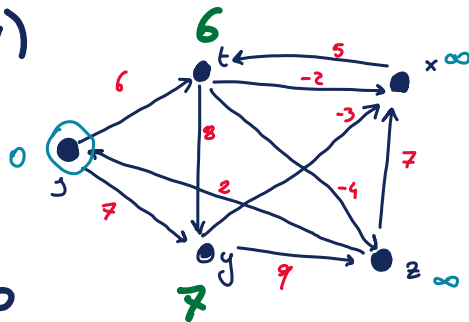
$$\begin{array}{ll} \varepsilon & \infty \\ (t,x) & -2 \quad x.d > t.d + (-2) \quad \text{NO} \end{array}$$

$$(t,y) \quad 2 \quad \text{NO}$$

$$(t,z) \quad -4 \quad \text{NO}$$

$$(x,t) \quad 5 \quad \text{NO}$$

$$(y,x) \quad -3 \quad \text{NO}$$

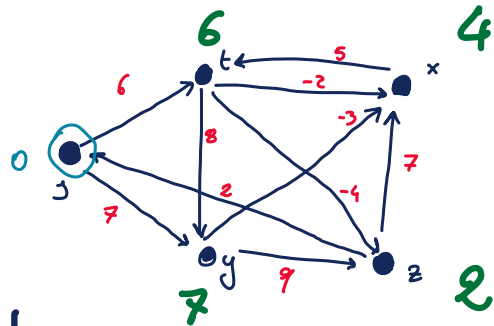


$(y, z)$  9 NO  
 $(z, x)$  7 NO  
 $(z, s)$  2  $s.d = 0 > z.d + 2$  NO  
 $(s, t)$  6  $t.d = \infty > s.d + 6 \Rightarrow t.d = 6, t.\pi = s$   
 $(s, y)$  7  $y.d = \infty > 0 + 7 \Rightarrow y.d = 7, y.\pi = s$

(2)

$\varepsilon$

$(t, x)$   $\omega$  -2  $\infty > 6 - 2 \rightarrow x.d = 4, x.\pi = t$   
 $(t, y)$  2  $7 > 6 + 2$  NO  
 $(t, z)$  -4  $\infty > 6 - 4 = 2 \rightarrow z.d = 2, z.\pi = t$   
 $(x, t)$  5  $6 > 4 + 5$  NO  
 $(y, x)$  -3  $4 > 7 - 3$  NO  
 $(y, z)$  9  $2 > 7 + 9$  NO  
 $(z, x)$  7  $4 > 2 + 7$  NO  
 $(z, s)$  2  $0 > 2 + 2 = 4$  NO  
 $(s, t)$  6  $6 > 0 + 6$  NO  
 $(s, y)$  7  $7 > 0 + 7$  NO

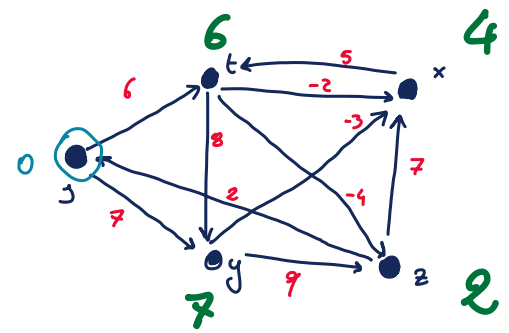


(3)

$\varepsilon$

$(t, x)$   $\omega$  -2  $4 > 6 - 2$  NO  
 $(t, y)$  2  $7 > 6 + 2$  NO  
 $(t, z)$  -4  $2 > 6 - 4$  NO

$(x, t)$  5       $6 > 4 + 5$  NO  
 $(y, x)$  -3       $4 > 7 - 3$  NO  
 $(y, z)$  9       $2 > 7 + 9$  NO  
 $(z, x)$  7       $4 > 2 + 7$  NO  
 $(z, s)$  2       $0 > 2 + 2$  NO  
 $(s, t)$  6       $6 > 0 + 6$   
 $(s, y)$  7       $7 > 0 + 7$



(4) Si ripete

TRUE  $\rightarrow$  la seconda parte di Bellman-Ford sarebbe uguale a (3)

$$\begin{array}{l|l}
 \delta(s, s) = 0 & s.\pi = NIL \\
 \delta(s, t) = 6 = t.d & t.\pi = s \\
 \delta(s, x) = 4 & x.\pi = t \\
 \delta(s, y) = 7 & y.\pi = s \\
 \delta(s, z) = 2 & z.\pi = t
 \end{array}$$

$$\delta(s, v) = v.d \quad \forall v$$

